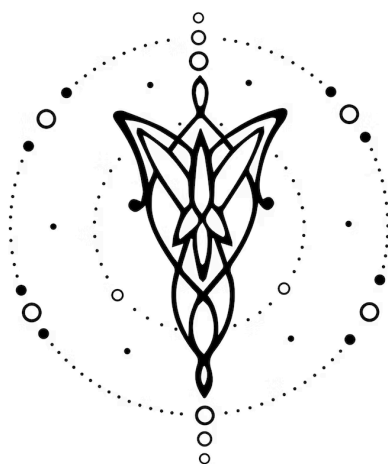




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE MUSIC DSL COMPILER

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Nicolás	Mazzitelli	62334	nmazzitelli@itba.edu.ar
Agustín	Brunero	62013	abrunero@itba.edu.ar
Mateo	Pirola	62810	mpirolapaulovich@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/nmazzitelli/music-dsl-compiler>.

3. Dominio

Desarrollar un lenguaje que permita crear y componer piezas musicales mediante código. El lenguaje debe permitir definir notas, acordes, silencios, ritmos, instrumentos y estructuras musicales como tracks y compases, para luego compilarlos en archivos MIDI ejecutables por cualquier reproductor compatible.

Para reducir la complejidad del proyecto, las composiciones se limitan a las funcionalidades del estándar General MIDI (128 instrumentos, 16 canales, 127 niveles de velocidad). No se contempla la síntesis de audio en tiempo real ni la generación de formatos de audio como WAV o MP3. El artefacto de salida es exclusivamente un archivo MIDI. Los tracks definidos en un programa se reproducen en simultáneo, cada uno asignado a un canal MIDI independiente, permitiendo composiciones con múltiples instrumentos.

Cada programa describe una composición que contiene uno o más tracks. Cada track está asociado a un instrumento y contiene una secuencia de eventos musicales, notas con altura y duración, acordes, silencios y repeticiones. Las propiedades globales como el tempo (BPM) y la signatura de tiempo definen el contexto rítmico de la composición.

La implementación satisfactoria de este lenguaje permitiría a compositores, estudiantes de música y desarrolladores prototipar ideas musicales rápidamente mediante texto, versionar sus composiciones con herramientas estándar de control de código, y automatizar la generación de piezas musicales paramétricas o algorítmicas.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrá definir el tempo global de la composición en BPM (beat per minute), por ejemplo *tempo 120*.
- (II). Se podrán definir la signatura de tiempo (time signature) de la composición, por ejemplo *time_signature 4/4*.
- (III). Se podrán crear uno o varios tracks, cada uno asociado a un instrumento MIDI específico mediante su nombre (por ejemplo, “*piano*”, “*violin*”, “*bass*”).
- (IV). Se podrán tocar notas individuales indicando su altura (nota + octava, por ejemplo, C4, D#5, Bb3) y su duracion (*whole, half, quarter, eighth, sixteenth*).
- (V). Se podrán tocar acordes, es decir, múltiples notas en simultáneo, indicando las notas entre corchetes y una duración.
- (VI). Se podrán definir silencios (rests) con una duración determinada.
- (VII). Se podrá especificar la velocidad (volumen/intensidad) de una nota o acorde mediante un modificador opcional.
- (VIII). Se proveerá una estructura de repetición (*repeat N { ... }*) para reiterar bloques de eventos musicales.
- (IX). Se proveerá una estructura condicional básica de tipo IF-THEN-ELSE.
- (X). Las variables podrán ser de tipo *integer, boolean, o string*.

- (XI). Se proveerán operadores aritméticos básicos (+, -, *, /) y relacionales (<, >, =, !=, <=, >=).
- (XII). Se proveerán operadores lógicos básicos como AND, OR y NOT.
- (XIII). Se podrán definir variables para almacenar valores numéricos y reutilizarlos como parámetros de duración, octava o repetición.
- (XIV). Se podrán agregar comentarios de línea con // y comentarios de bloque con /* ... */.
- (XV). Se podrá definir opcionalmente la tonalidad de la composición mediante la palabra reservada *key*, indicando la nota fundamental y el modo (mayor o menor), por ejemplo *key G major* o *key A minor*. Al definir una tonalidad, las alteraciones propias de la escala se aplican automáticamente a las notas escritas (por ejemplo, en *key G major*, toda nota F se interpreta como F#). Si se desea anular la alteración, se puede indicar explícitamente con el modificador *n* (natural), por ejemplo *Fn4*. El uso de *key* es opcional, sin esta declaración, cada nota suena exactamente como se escribe.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que defina un tempo y un track con una secuencia de notas simples.
- (II). Un programa que utilice múltiples tracks con diferentes instrumentos.
- (III). Un programa que defina y toque acordes.
- (IV). Un programa que utilice silencios (rests) entre notas.
- (V). Un programa que utilice la estructura repeat para repetir un patrón.
- (VI). Un programa que defina variables y las utilice como parámetros.
- (VII). Un programa que utilice diferentes duraciones de notas en un mismo track.
- (VIII). Un programa que defina la signature de tiempo y el tempo.
- (IX). Un programa que utilice modificadores de velocidad en las notas.
- (X). Un programa que combine tracks, acordes, repeticiones y silencios en una composición completa.
- (XI). Un programa que defina una tonalidad con *key* y utilice notas dentro de esa escala.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado sintácticamente (por ejemplo, falta de punto y coma o llaves desbalanceadas).
- (II). Un programa que referencie un nombre de instrumento inválido.
- (III). Un programa que utilice una nota con octava fuera de rango (por ejemplo, C10).
- (IV). Un programa que defina dos tracks con el mismo nombre.
- (V). Un programa que intente operar entre tipos de datos incompatibles (por ejemplo, sumar un string y un integer).
- (VI). Un programa que utilice el modificador natural (*n*) sin haber definido una tonalidad con *key*.

6. Ejemplos

En el **Cód. (6.1)**, se puede ver la creación de una composición simple con un track de piano que toca una escala de Do mayor, demostrando el uso de notas individuales con diferentes duraciones.

```
// Composición simple: escala de Do mayor
tempo 120;
time_signature 4/4;
key C major;

track escala instrument "piano" {
    play C4 quarter;
    play D4 quarter;
    play E4 quarter;
    play F4 quarter;
    play G4 quarter;
    play A4 quarter;
    play B4 quarter;
    play C5 whole;
}
```

Código 6.1: Escala de Do mayor en piano.

En el **Cód. (6.2)**, se muestra una composición más compleja con múltiples tracks, acordes, silencios y uso de la estructura repeat para crear un patrón rítmico repetitivo.

```
// Composición con múltiples tracks y acordes
tempo 100;
time_signature 4/4;
key A minor;

track melodia instrument "piano" {
    play [C4, E4, G4] half;    // Acorde de Do mayor
    rest quarter;
    play E4 eighth;
    play D4 eighth;

    repeat 2 {
        play C4 quarter;
        play G4 quarter;
    }
}

track bajo instrument "bass" {
    play C2 whole;
}
```

```
    play G2 whole;
    rest half;
    play A2 half;
}

track ritmo instrument "drums" {
    repeat 4 {
        play C3 quarter velocity 100;
        play C3 eighth velocity 60;
        play C3 eighth velocity 60;
    }
}
```

Código 6.2: Composición con múltiples tracks, acordes, silencios y repeticiones.